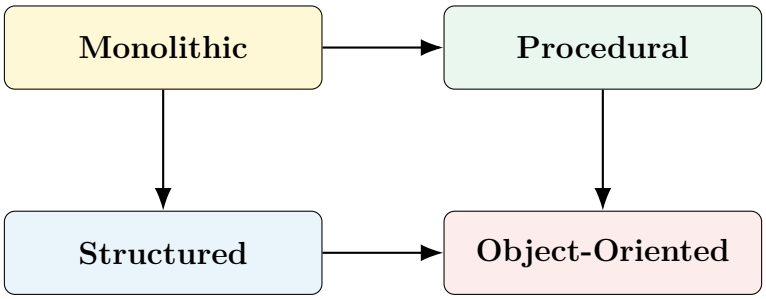
OBJECT ORIENTED PROGRAMMING SYSTEM

Course Code: 25CS301

Lecture 1

Evolution of Programming Paradigms and Introduction to Object-Oriented Programming



Prepared for B.Tech III Semester

Duration: 60 minutes

Based on Unit 1 of the autonomous syllabus

Contents

1. Lecture Overview

1.1 Position of this lecture in Unit 1

The first unit begins with the evolution of programming paradigms and then moves towards the structure of a C++ program, classes and objects, scope resolution operator, reference variables, functions, parameter passing, inline functions, overloading, default arguments, variadic functions, the `auto` keyword and range-based loops.

This lecture deliberately concentrates on the conceptual foundation. Students must first understand *why* object-oriented programming became necessary before they begin writing classes and objects in C++.

1.2 Learning outcomes

By the end of this lecture, a student should be able to:

1. define a programming paradigm;
2. explain the evolution from monolithic to object-oriented development;
3. distinguish procedural and structured programming;
4. identify the limitations of function-centred software design;
5. explain the need for objects, classes and data protection;
6. compare structured development and object-oriented development;
7. identify situations where OOP is useful and situations where it may be unnecessary;
8. explain why C++ is widely used for object-oriented and systems programming.

1.3 Suggested 60-minute teaching plan

Time	Activity
0–5 min	Introduce the course, ask students how large applications are organised, and state the learning outcomes.
5–15 min	Explain programming paradigms and why programming styles evolve.
15–25 min	Explain monolithic, procedural and structured programming with diagrams.
25–37 min	Discuss the limitations of structured programming in large systems.

37–50 min	Introduce object-oriented thinking, class, object, state and behaviour.
50–56 min	Compare structured and object-oriented development.
56–60 min	Recap, quick questions and assignment.

Teacher's Note

Do not begin the course by asking students to memorise the four pillars of OOP. First establish the engineering problem: large software is difficult to organise, protect, test and extend. OOP becomes meaningful only after that problem is clear.

2. What Is a Programming Paradigm?

Definition

A **programming paradigm** is a general style, model or approach used to organise the logic, data and control flow of a computer program.

A paradigm influences the following questions:

- How is a large problem divided into smaller parts?
- Where is data stored?
- Which part of the program is allowed to modify that data?
- How are program components connected?
- How can existing code be reused?
- How can changes be made without breaking unrelated features?

A paradigm is not merely a programming language. The same language may support more than one paradigm. C++, for example, supports procedural programming, object-oriented programming and generic programming.

Example

A travel problem can be solved by travelling by bus, train, car or aircraft. The destination may be the same, but the method differs. Similarly, the required output of a program may remain the same while the method used to organise the program changes.

2.1 Why paradigms evolve

Software evolves in size and complexity. A program of 50 lines can be understood by one person. A program containing millions of lines, several databases, multiple user interfaces and many developers cannot be managed in the same way.

Programming paradigms evolved to improve:

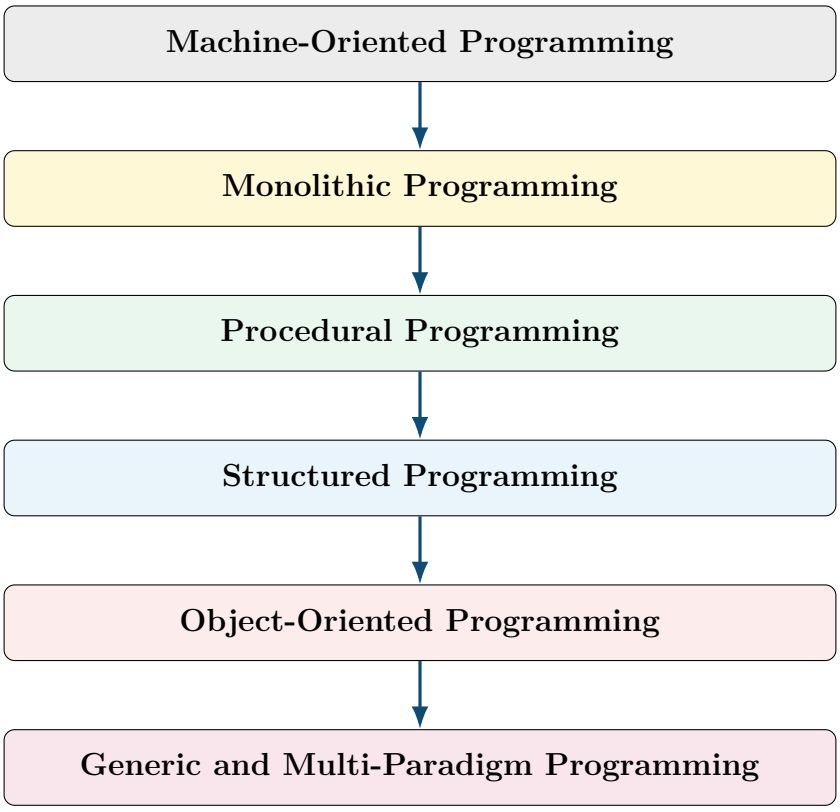
- readability;
- modularity;
- reliability;
- security;
- reuse;
- maintainability;

- scalability;
- team development.

Remember

Paradigm evolution does not mean that every old paradigm becomes useless. Older approaches may still be effective for small, specialised or performance-critical programs.

3. Evolution of Programming Approaches



3.1 Machine-oriented beginnings

Early computers were programmed close to the hardware. Instructions were written in machine code or assembly language. Programmers had to manage registers, memory addresses and hardware-specific instructions directly.

Major limitations:

- difficult to read and write;
- highly dependent on hardware;
- difficult to debug;
- unsuitable for large teams;
- very low portability.

High-level programming languages reduced these difficulties by allowing programmers to express algorithms in a more human-readable form.

Teacher's Note

The purpose of discussing machine language is not to teach instruction encoding. The purpose is to show that programming gradually moved from hardware-centred thinking to problem-centred thinking.

4. Monolithic Programming

Definition

In **monolithic programming**, the complete program is written as one large, tightly connected block with little or no logical decomposition.

4.1 Basic structure

Single Program Block

Input statements
Validation logic
Calculations
Display statements
File handling
Error handling
More calculations
Final output

4.2 Characteristics

- most logic is placed in one main program;
- data is often global or widely accessible;
- there is little separation of responsibility;
- one change may affect many unrelated statements;
- testing individual portions is difficult.

4.3 Advantages

- quick to write for extremely small programs;
- low design overhead;
- suitable for simple demonstrations and one-time scripts.

4.4 Disadvantages

- poor readability;
- difficult debugging;
- difficult testing;
- almost no reuse;
- high risk of side effects;
- unsuitable for teamwork;
- maintenance cost increases rapidly as the program grows.

Example

Imagine a 1000-page book without chapters, headings or an index. The book may contain all required information, but locating, correcting or replacing one topic becomes difficult. A monolithic program suffers from the same organisational problem.

Common Mistake

Do not describe every program containing a `main()` function as monolithic. A C++ program always has an entry point, but it may still be modular if responsibilities are divided among functions, classes and files.

5. Procedural Programming

Definition

Procedural programming organises a program as a collection of procedures or functions. Each function performs a specific operation, and larger tasks are achieved by calling several functions in sequence.

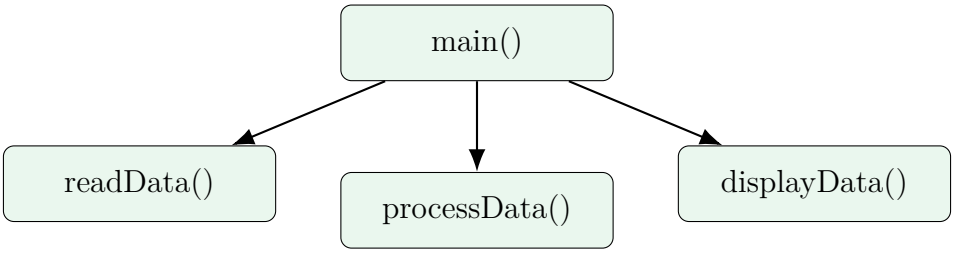
5.1 Function-centred design

In procedural programming, the central question is:

> *Which sequence of operations must be performed to solve the problem?*

The program is decomposed into functions such as:

- `readData()`
- `validateData()`
- `calculateResult()`
- `displayResult()`



5.2 Simple procedural example

```
#include <iostream>
using namespace std;

void readMarks(int &m1, int &m2, int &m3) {
    cin >> m1 >> m2 >> m3;
}

int calculateTotal(int m1, int m2, int m3) {
    return m1 + m2 + m3;
}

```

```
12 void displayResult(int total) {
13     cout << "Total = " << total << '\n';
14 }
15
16 int main() {
17     int mark1, mark2, mark3;
18     readMarks(mark1, mark2, mark3);
19     int total = calculateTotal(mark1, mark2, mark3);
20     displayResult(total);
21     return 0;
22 }
```

Listing 1: Procedural program for student result processing

5.3 Strengths

- simpler than monolithic programming;
- supports modular functions;
- individual functions can be tested;
- common operations can be reused;
- natural for algorithmic and mathematical problems.

5.4 Limitations

- functions and data are usually treated separately;
- shared data may be modified by many functions;
- large systems may contain hundreds of tightly coupled functions;
- data protection is weak when global variables are used;
- mapping real-world entities to independent functions is often unnatural.

6. Structured Programming

Definition

Structured programming is a disciplined form of procedural programming that uses well-defined control structures, modular decomposition and restricted flow of control to improve clarity and reliability.

6.1 Three basic control structures

Every structured algorithm can be constructed from the following:

6.1.1 Sequence

Statements are executed one after another.

```
input = readValue();
result = input * input;
cout << result;
```

6.1.2 Selection

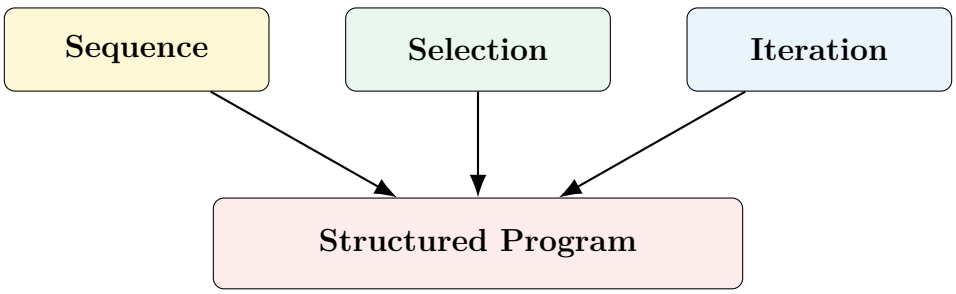
One among several alternatives is selected.

```
if (marks >= 40)
    cout << "Pass";
else
    cout << "Fail";
```

6.1.3 Iteration

A group of statements is repeated.

```
for (int i = 1; i <= 10; ++i)
    cout << i << ' ';
```



6.2 Top-down design

Structured programs usually use top-down decomposition:

1. start with the complete problem;
2. divide it into major tasks;
3. divide each task into smaller sub-tasks;
4. continue until every task can be implemented as a function.

Example

For a library system, a top-down design may first identify issue book, return book and search book. The issue-book operation may then be divided into check membership, check availability, update stock and generate receipt.

6.3 Benefits

- clearer control flow;
- reduced use of unrestricted jumps;
- easier testing and debugging;
- improved modularity;
- better documentation;
- suitable for many small and medium-sized applications.

6.4 Persistent limitations

Structured programming improves control flow but does not automatically solve the problem of data ownership. A function may still receive or modify data that logically belongs to another part of the program.

Remember

Structured programming primarily organises **operations**. Object-oriented programming organises both **data and operations** around objects.

7. Why Function-Centred Design Becomes Difficult

Consider a banking application containing customer data, account balances, loans, transactions, cards and audit records. A purely function-centred design may contain functions such as:

`deposit()`, `withdraw()`, `transfer()`, `updateCustomer()`, `closeAccount()`, `calculateInterest()`, `generateStatement()`

The difficulty is not the number of functions alone. The real difficulty is controlling access to shared data.

7.1 Example of unsafe shared data

```cpp
#include <iostream>
using namespace std;

double balance = 10000.0;    // globally accessible

void withdraw(double amount) {
    balance -= amount;
}

void resetForTesting() {
    balance = 0.0;            // accidentally used in production
}

int main() {
    withdraw(500.0);
    resetForTesting();
    cout << balance;
}
```

Listing 2: Global data can be modified from unrelated functions

7.2 Typical problems in large procedural systems

1. **Weak data protection:** many functions can modify the same data.
2. **High coupling:** changing a data format may require changing many functions.
3. **Low cohesion:** operations related to one entity may be scattered across files.
4. **Naming conflicts:** large projects contain many similarly named variables and functions.

5. **Maintenance difficulty:** developers struggle to determine which function owns which responsibility.
6. **Limited extensibility:** adding a new entity type often requires modifying existing logic.

Common Mistake

The conclusion is not that procedural programming is always bad. The conclusion is that a function-centred approach becomes difficult when the software contains many interacting entities with long-lived state.

8. The Need for Object-Oriented Programming

Object-oriented programming emerged to support the organisation of complex software around entities that contain both data and behaviour.

8.1 Engineering requirements addressed by OOP

Requirement	How object-oriented design helps
Data protection	Restricts direct access and exposes controlled operations.
Local reasoning	Related data and functions are placed together.
Modularity	Each class represents a logical component.
Reusability	Existing classes can be reused, composed or extended.
Maintainability	Changes can often be restricted to one class.
Scalability	Large applications can be organised as collaborating objects.
Teamwork	Different teams can work on separate classes and interfaces.
Real-world modelling	Software components resemble real entities and concepts.

8.2 The central shift in thinking

Procedural thinking asks:

> What steps must be performed?

Object-oriented thinking asks:

> Which objects participate, what information does each object own, and what responsibilities does each object perform?

Example

For an online shopping system, object-oriented analysis may identify Customer, Product, Cart, Order, Payment and Shipment objects. Each object owns relevant information and provides relevant operations.

9. Fundamental Object-Oriented Vocabulary

9.1 Object

Definition

An **object** is an identifiable software entity that has state, behaviour and identity.

State describes the current data of the object.
Behaviour describes the operations the object can perform.
Identity distinguishes one object from another object even when their data is similar.

Example

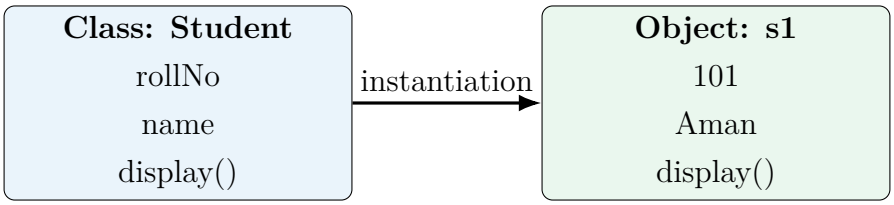
A bank account object may have account number, account holder and balance as state. It may provide deposit, withdraw and display-balance operations as behaviour.

9.2 Class

Definition

A **class** is a user-defined blueprint or template that describes the common data members and member functions of a category of objects.

A class is a design. An object is a usable instance created from that design.



9.3 Attribute and method

An **attribute** is data associated with an object. In C++, attributes are generally represented by data members.

A **method** is an operation associated with an object. In C++, methods are represented by member functions.

9.4 Message passing

Objects collaborate by requesting operations from one another. Conceptually, this interaction is called message passing. In C++, it commonly appears as a member function call such as:

```
account.deposit(5000.0);
```

10. First Class and Object Example in C++

The detailed syntax of classes and objects belongs to the next part of Unit 1, but a preview helps students connect the concept to code.

```cpp
#include <iostream>
#include <string>
using namespace std;

class Student {
private:
    int rollNumber;
    string name;

public:
    void setData(int roll, const string &studentName) {
        rollNumber = roll;
        name = studentName;
    }

    void display() const {
        cout << "Roll Number: " << rollNumber << '\n';
        cout << "Name: " << name << '\n';
    }
};

int main() {
    Student s1;                // object creation
    s1.setData(101, "Aman");
    s1.display();
    return 0;
}
```

Listing 3: A basic Student class

10.1 Reading the program conceptually

1. `Student` is a class.
2. `s1` is an object of the class.
3. `rollNumber` and `name` represent state.
4. `setData()` and `display()` represent behaviour.

5. `private` data cannot be accessed directly from `main()`.
6. `public` functions provide controlled interaction with the object.

Common Mistake

A class is not an object. A class defines a category. Objects are individual instances created from the class.

11. Introductory View of the Four Major OOP Concepts

These concepts will be studied in detail in later units. At this stage, students need only a correct conceptual preview.

11.1 Encapsulation

Encapsulation means combining data and the operations that work on that data into one unit, usually a class. It also supports controlled access.

Example

A bank account should not allow arbitrary code to assign a negative balance. Instead, the account class should provide controlled operations such as deposit and withdraw.

11.2 Abstraction

Abstraction means exposing essential operations while hiding unnecessary implementation details.

A person can drive a car using the steering wheel, brake and accelerator without knowing the detailed design of the engine control system.

11.3 Inheritance

Inheritance allows a new class to acquire or extend the characteristics of an existing class.

For example, SavingsAccount and CurrentAccount may share common account information while providing specialised behaviour.

11.4 Polymorphism

Polymorphism means one interface can represent different forms of behaviour. A function such as `draw()` may behave differently for Circle, Rectangle and Triangle objects.

Remember

The four concepts are connected but not identical. Encapsulation organises and protects data; abstraction hides complexity; inheritance supports extension; polymorphism supports substitutable behaviour.

12. Structured Development vs Object-Oriented Development

Basis	Structured Development	Object-Oriented Development
Primary focus	Functions and algorithms	Objects and responsibilities
Approach	Usually top-down	Usually bottom-up or responsibility-driven
Decomposition	Divides the problem into functions	Divides the problem into collaborating objects
Data treatment	Data and functions are often separate	Data and related operations are combined
Data protection	Depends on programmer discipline	Supported through access control
Reusability	Function reuse	Class, component, inheritance and composition reuse
Change impact	Data changes may affect many functions	Change can often be localised within a class
Real-world mapping	Indirect	More direct for entity-rich systems
Testing	Function-oriented testing	Class and object behaviour testing
Suitable for	Algorithms, utilities, small systems	Large, evolving, entity-rich applications
Examples	Numerical computation, data conversion	Banking, e-commerce, games, GUI systems

12.1 Top-down and bottom-up thinking

Top-down design starts with the complete task and repeatedly divides it into smaller operations.

Bottom-up design starts by identifying reusable components or classes and then combines them to build the larger system.

Modern software development often uses a mixture of both. The distinction is useful for understanding emphasis, not for creating an absolute rule.

Common Mistake

Do not write that object-oriented design never uses functions or never uses top-down analysis. Member functions are still functions, and high-level decomposition is still necessary. The difference lies in how responsibilities and data ownership are organised.

13. Case Study: Student Record System

13.1 Structured view

A procedural design may use a structure for data and several independent functions:

```
struct Student {
    int rollNumber;
    string name;
    double marks;
};

void readStudent(Student &s);
void displayStudent(const Student &s);
double calculateGrade(const Student &s);
```

This design can be effective, especially for a small program. However, any function receiving a non-constant reference may change the student's data.

13.2 Object-oriented view

```
class Student {
private:
    int rollNumber;
    string name;
    double marks;

public:
    void setDetails(int roll, const string &n, double m);
    void display() const;
    char grade() const;
};
```

The class declares which information is internal and which operations are available to users of the class.

13.3 Comparison of responsibility

- In the structured design, functions operate on a student record.
- In the object-oriented design, the Student object is responsible for maintaining and presenting its own valid state.

Teacher's Note

Use this case study to show that OOP is not merely replacing `struct` with `class`. The real change is ownership of responsibility and controlled access.

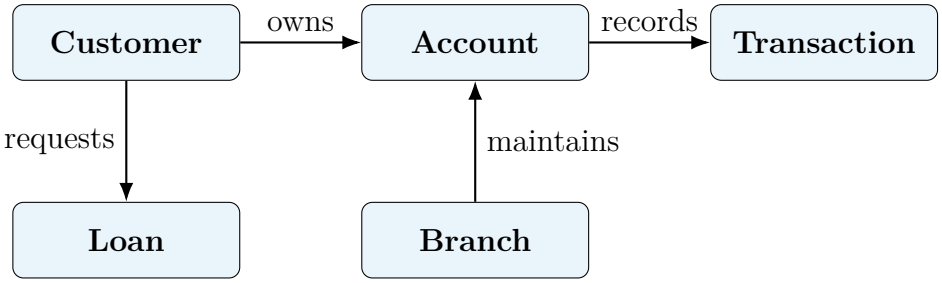
14. Case Study: Banking System

14.1 Required entities

A banking system may contain:

- Customer
- Account
- Transaction
- Branch
- Loan
- Employee
- Card

14.2 Possible object relationships



14.3 Why OOP helps

- Account protects its balance.
- Transaction records immutable transaction details.
- Customer manages personal and contact information.
- Loan applies loan-specific rules.
- Each class can be tested independently.
- New account types can be added with limited changes.

14.4 Controlled account behaviour

```
class BankAccount {
private:
    double balance = 0.0;

public:
    bool deposit(double amount) {
        if (amount <= 0.0)
            return false;
        balance += amount;
        return true;
    }

    bool withdraw(double amount) {
        if (amount <= 0.0 || amount > balance)
            return false;
        balance -= amount;
        return true;
    }

    double getBalance() const {
        return balance;
    }
};
```

15. Benefits of Object-Oriented Programming

15.1 Modularity

A class groups related data and behaviour. The internal implementation of one class can often be changed without rewriting the complete application.

15.2 Data protection

Access specifiers such as `private`, `protected` and `public` help control which program components may access class members.

15.3 Maintainability

When each class has a clear responsibility, defects and feature changes can be located more easily.

15.4 Reusability

Reusable classes and components reduce duplication. Reuse may be achieved through composition, templates, libraries or inheritance.

15.5 Extensibility

New behaviour can often be added through new classes instead of repeatedly changing stable code.

15.6 Natural modelling

Classes can represent entities such as Student, Product, Sensor, Vehicle, User, Invoice and Course.

15.7 Team development

Interfaces between classes allow different developers to work on separate modules.

Remember

OOP does not automatically produce good software. Poorly designed classes can create excessive coupling, deep inheritance hierarchies and unnecessary complexity. Design quality still matters.

16. Limitations and Misuse of OOP

16.1 Possible disadvantages

- additional design effort for very small programs;
- memory and performance overhead in some designs;
- excessive abstraction can make code difficult to follow;
- poorly chosen class boundaries increase coupling;
- inheritance can be misused;
- beginners may create classes without meaningful responsibilities.

16.2 When a simple procedural solution may be better

- a short mathematical calculation;
- a small file-conversion utility;
- a simple command-line script;
- a performance-critical routine with no long-lived state;
- an algorithm whose natural structure is function-oriented.

16.3 Balanced conclusion

The correct question is not “Is OOP always better?” The correct question is “Which design approach best matches the structure, size and expected evolution of the problem?”

Common Mistake

Creating a class for every variable or every function is not object-oriented design. A useful class should represent a coherent concept and should have a clear responsibility.

17. Why C++ for Object-Oriented Programming?

C++ was developed by Bjarne Stroustrup as an extension of the C language. It combines low-level control and high performance with high-level abstraction mechanisms.

17.1 Important characteristics

- supports procedural, object-oriented and generic programming;
- provides classes, objects, constructors and destructors;
- supports function and operator overloading;
- supports inheritance and runtime polymorphism;
- provides templates and the Standard Template Library;
- offers deterministic resource management;
- allows close interaction with hardware;
- produces efficient native code.

17.2 Application areas

- operating systems and device drivers;
- game engines and graphics;
- embedded systems and robotics;
- financial and low-latency software;
- browsers and compilers;
- scientific and engineering applications;
- real-time systems;
- database and networking infrastructure.

17.3 C++ is a multi-paradigm language

A C++ programmer may use:

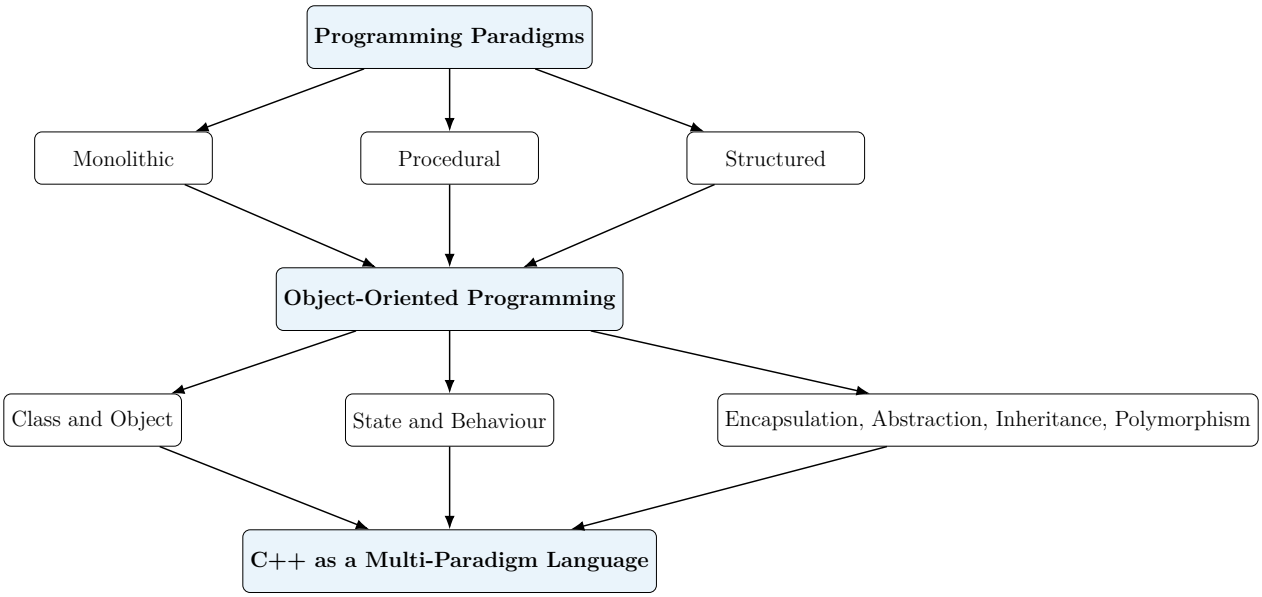
- procedural functions for simple operations;
- classes for stateful entities;
- templates for generic algorithms;

- lambda expressions for local callable behaviour;
- standard containers and algorithms for reusable data processing.

Remember

C++ does not force every program to be object-oriented. It allows the programmer to combine paradigms according to the problem.

18. Concept Map for Lecture 1



19. Common Misconceptions

1. **Misconception:** OOP means only using classes.
   Correction: Good OOP requires meaningful responsibilities, controlled state and clear collaboration.
2. **Misconception:** Every real-world noun must become a class.
   Correction: Only concepts relevant to the software model should become classes.
3. **Misconception:** OOP is always more efficient.
   Correction: OOP may improve development and maintenance, but poor designs can increase runtime and design overhead.
4. **Misconception:** Structured programming has no place in C++.
   Correction: C++ supports multiple paradigms. Structured functions remain useful.
5. **Misconception:** Encapsulation and abstraction are identical.
   Correction: Encapsulation groups and controls access; abstraction exposes essential behaviour and hides details.
6. **Misconception:** Inheritance is the main method of reuse.
   Correction: Composition and generic programming are often safer and more flexible.

20. Quick Revision Notes

One-Page Recap

- A programming paradigm is a style of organising programs.
- Monolithic programs place most logic in one large block.
- Procedural programs divide work into functions.
- Structured programming uses sequence, selection, iteration and modular design.
- Function-centred systems become difficult when many functions share and modify data.
- OOP organises software around objects containing state and behaviour.
- A class is a blueprint; an object is an instance.
- State is represented by data; behaviour is represented by operations.
- Encapsulation combines data and functions and supports controlled access.
- Abstraction hides unnecessary implementation details.
- Inheritance supports extension of existing classes.
- Polymorphism allows one interface to represent different behaviours.
- Structured development is mainly function-oriented; OOP is mainly responsibility-oriented.
- C++ supports procedural, object-oriented and generic programming.
- OOP is useful for large, evolving and entity-rich systems, but it is not mandatory for every problem.

21. Viva and Short-Answer Questions

1. What is a programming paradigm?
2. Why did programming paradigms evolve?
3. What is monolithic programming?
4. State two disadvantages of monolithic programming.
5. What is procedural programming?
6. Why is procedural programming called function-oriented?
7. What are the three basic control structures of structured programming?
8. What is top-down design?
9. What limitation of structured programming encouraged OOP?
10. Define object.
11. Define class.
12. Differentiate between class and object.
13. What is object state?
14. What is object behaviour?
15. What is encapsulation?
16. What is abstraction?
17. What is inheritance?
18. What is polymorphism?
19. Why is C++ called a multi-paradigm language?
20. Give two applications where OOP is useful.
21. Can procedural programming be used in C++?
22. Is OOP always better than procedural programming? Justify briefly.

22. Multiple-Choice Questions

1. A programming paradigm is best described as:
   (a) a compiler error
   (b) a style of organising programs
   (c) a hardware device
   (d) a database table
2. Structured programming primarily uses:
   (a) unrestricted jumps only
   (b) sequence, selection and iteration
   (c) objects only
   (d) machine instructions only
3. The main unit of decomposition in procedural programming is:
   (a) object
   (b) function
   (c) database
   (d) thread
4. An object contains:
   (a) only data
   (b) only functions
   (c) state and behaviour
   (d) only memory addresses
5. A class is:
   (a) an instance of an object
   (b) a blueprint for objects
   (c) a machine instruction
   (d) an operating system
6. Which concept mainly supports controlled access to data?
   (a) encapsulation
   (b) iteration
   (c) recursion

(d) compilation

7. Which approach usually emphasises top-down decomposition?

   (a) structured development

   (b) hardware design

   (c) object instantiation

   (d) garbage collection

8. C++ supports:

   (a) only procedural programming

   (b) only object-oriented programming

   (c) procedural, object-oriented and generic programming

   (d) no programming paradigm

9. Which system is most naturally modelled using interacting objects?

   (a) a single addition operation

   (b) banking application

   (c) one-line text output

   (d) simple temperature conversion

10. Which statement is correct?

    (a) OOP is always the best approach.

    (b) Procedural programming is never used in C++.

    (c) OOP helps organise stateful, evolving software.

    (d) Every variable should be a class.

Answer key

1-(b), 2-(b), 3-(b), 4-(c), 5-(b), 6-(a), 7-(a), 8-(c), 9-(b), 10-(c)

23. Descriptive and Examination-Oriented Questions

23.1 Two-mark questions

1. Define programming paradigm.
2. List the basic control structures of structured programming.
3. Define class and object.
4. State two advantages of OOP.
5. Why is C++ called a multi-paradigm language?

23.2 Five-mark questions

1. Explain monolithic and procedural programming with suitable examples.
2. Discuss the main characteristics of structured programming.
3. Explain the need for object-oriented programming.
4. Differentiate between class and object with a real-world example.
5. Explain the introductory concepts of encapsulation and abstraction.

23.3 Ten-mark questions

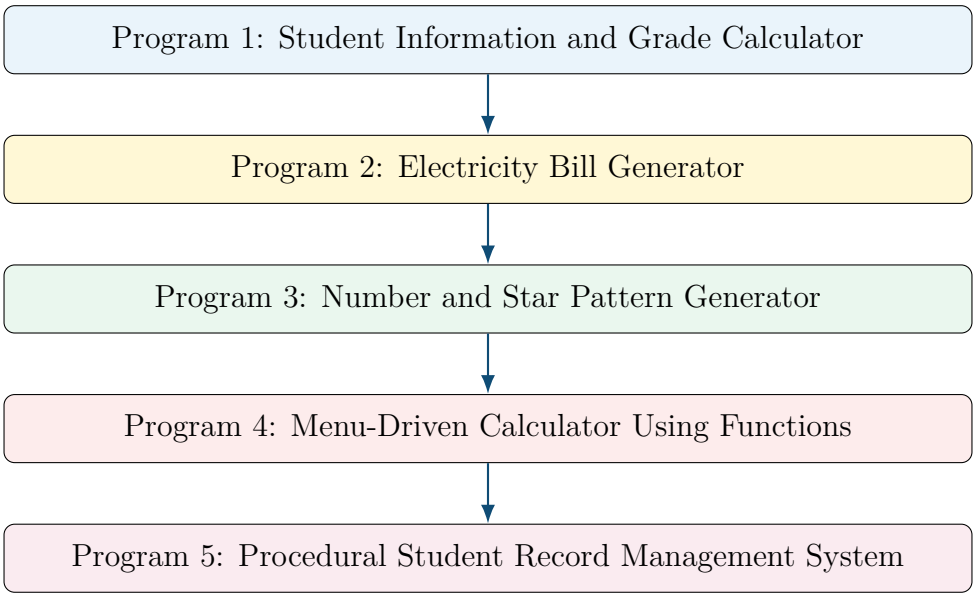
1. Explain the evolution of programming paradigms from monolithic programming to object-oriented programming. Include advantages and limitations of each stage.
2. Compare structured development and object-oriented development on the basis of decomposition, data security, reusability, maintainability and suitability.
3. Using a banking or student-management case study, explain how an object-oriented design improves data ownership and modularity.

A. Foundation Programs for Learning C++

Teacher's Note

These programs are deliberately arranged from basic syntax to modular problem solving. Students should type, compile, execute and modify every program. Merely reading the code will not build programming ability.

A.1 Recommended Learning Sequence



Remember

The fifth program is a bridge to object-oriented programming. After students complete the procedural version, the same system can be redesigned using a `Student` class in the next lecture or laboratory session.

A.2 Program 1: Student Information and Grade Calculator

A.2.1 Problem statement

Write a C++ program that accepts a student's name, roll number, branch and marks in five subjects. Calculate the total marks, percentage and grade, and display a formatted result.

A.2.2 Concepts covered

- Header files and the `main()` function
- Variables and fundamental data types
- `string`, `cin`, `getline()` and `cout`
- Arithmetic operators
- Conditional statements
- Type conversion and formatted output

A.2.3 Algorithm

1. Read name, roll number and branch.
2. Read marks in five subjects.
3. Add the marks to calculate the total.
4. Divide the total by five to obtain the percentage.
5. Assign a grade using an `if-else-if` ladder.
6. Display all details in a formatted result sheet.

```
#include <iomanip>
#include <iostream>
#include <string>
using namespace std;

int main() {
    string name;
    string branch;
    int rollNumber;
    double m1, m2, m3, m4, m5;

    cout << "Enter student name: ";
    getline(cin, name);
```

```
14
15      cout << "Enter roll number: ";
16      cin >> rollNumber;
17      cin.ignore();
18
19      cout << "Enter branch: ";
20      getline(cin, branch);
21
22      cout << "Enter marks in five subjects: ";
23      cin >> m1 >> m2 >> m3 >> m4 >> m5;
24
25      const double total = m1 + m2 + m3 + m4 + m5;
26      const double percentage = total / 5.0;
27      char grade;
28
29      if (percentage >= 90.0) {
30          grade = 'A';
31      } else if (percentage >= 80.0) {
32          grade = 'B';
33      } else if (percentage >= 70.0) {
34          grade = 'C';
35      } else if (percentage >= 60.0) {
36          grade = 'D';
37      } else {
38          grade = 'F';
39      }
40
41      cout << "\n---------- RESULT ----------\n";
42      cout << "Name        : " << name << '\n';
43      cout << "Roll No.    : " << rollNumber << '\n';
44      cout << "Branch      : " << branch << '\n';
45      cout << "Total       : " << total << " / 500\n";
46      cout << fixed << setprecision(2);
47      cout << "Percentage  : " << percentage << "%\n";
48      cout << "Grade       : " << grade << '\n';
49
50      return 0;
51  }
```

Listing 4: Student information and grade calculator

A.2.4 Sample execution

```
Enter student name: Riya Sharma
Enter roll number: 105
Enter branch: CSE
Enter marks in five subjects: 86 91 78 88 84

---------- RESULT ----------
Name       : Riya Sharma
Roll No.   : 105
Branch     : CSE
Total      : 427 / 500
Percentage : 85.40%
Grade      : B
```

Common Mistake

After using `cin >> rollNumber`, a newline remains in the input buffer. Calling `getline()` immediately may read that newline as an empty string. The statement `cin.ignore()` removes it.

A.2.5 Practice extensions

1. Reject marks outside the range 0 to 100.
2. Display `PASS` only if the student obtains at least 40 marks in every subject.
3. Use an array and loop instead of five separate variables.

A.3 Program 2: Electricity Bill Generator

A.3.1 Problem statement

Calculate an electricity bill using the following progressive slabs:

Consumption slab	Rate per unit
First 100 units	Rs. 5.00
Next 100 units	Rs. 7.00
Units above 200	Rs. 10.00

Add 18% tax to the energy charge and display the complete bill.

A.3.2 Important clarification

The rates are progressive. For 250 units, the complete consumption is not charged at Rs. 10 per unit. Instead:

$$(100 \times 5) + (100 \times 7) + (50 \times 10)$$

A.3.3 Algorithm

1. Read the number of consumed units.
2. Reject negative input.
3. Calculate the energy charge according to progressive slabs.
4. Calculate tax as 18% of the energy charge.
5. Add energy charge and tax to obtain the final amount.

```
#include <iomanip>
#include <iostream>
using namespace std;

int main() {
    double units;
    cout << "Enter electricity units consumed: ";
    cin >> units;

    if (units < 0) {
        cerr << "Error: units cannot be negative.\n";
        return 1;
    }
```

```
14
15      double energyCharge = 0.0;
16
17      if (units <= 100) {
18          energyCharge = units * 5.0;
19      } else if (units <= 200) {
20          energyCharge = (100 * 5.0)
21                       + ((units - 100) * 7.0);
22      } else {
23          energyCharge = (100 * 5.0)
24                       + (100 * 7.0)
25                       + ((units - 200) * 10.0);
26      }
27
28      const double tax = energyCharge * 0.18;
29      const double finalAmount = energyCharge + tax;
30
31      cout << fixed << setprecision(2);
32      cout << "\n------- ELECTRICITY BILL -------\n";
33      cout << "Units consumed : " << units << '\n';
34      cout << "Energy charge  : Rs. " << energyCharge << '\n';
35      cout << "Tax (18%)      : Rs. " << tax << '\n';
36      cout << "Final amount   : Rs. " << finalAmount << '\n';
37
38      return 0;
39  }
```

Listing 5: Electricity bill generator using an if–else-if ladder

A.3.4 Dry run for 250 units

$$\begin{aligned}\text{Energy charge} &= (100 \times 5) + (100 \times 7) + (50 \times 10)\\ &= 500 + 700 + 500\\ &= \text{Rs. } 1700\\ \text{Tax} &= 1700 \times 0.18 = \text{Rs. } 306\\ \text{Final amount} &= 1700 + 306 = \text{Rs. } 2006\end{aligned}$$

Common Mistake

Do not write `energyCharge = units * 10` for every value above 200. That implements a flat slab, not a progressive slab.

A.3.5 Practice extensions

1. Add a fixed meter charge of Rs. 75.
2. Give a 5% rebate when payment is made before the due date.
3. Print a warning when consumption exceeds 500 units.

A.4 Program 3: Number and Star Pattern Generator

A.4.1 Purpose

Pattern programs are not important because industries print triangles of stars. They are useful because they force students to understand loop control, nested repetition, row-column relationships and tracing.

A.4.2 Pattern A: Increasing number triangle

For $n = 5$, print:

```
1
12
123
1234
12345
```

```cpp
#include <iostream>
using namespace std;

int main() {
    int rows;
    cout << "Enter number of rows: ";
    cin >> rows;

    if (rows <= 0) {
        cerr << "Rows must be positive.\n";
        return 1;
    }

    for (int i = 1; i <= rows; ++i) {
        for (int j = 1; j <= i; ++j) {
            cout << j;
        }
        cout << '\n';
    }

    return 0;
}
```

Listing 6: Increasing number triangle

A.4.3 Loop interpretation

- The outer loop controls the row number.
- The inner loop controls how many values appear in that row.
- In row i, the inner loop runs exactly i times.

A.4.4 Pattern B: Centred pyramid

For $n = 5$, print:

```
    *
   ***
  *****
 *******
*********
```

```cpp
#include <iostream>
using namespace std;

int main() {
    int rows;
    cout << "Enter number of rows: ";
    cin >> rows;

    for (int i = 1; i <= rows; ++i) {
        for (int space = 1; space <= rows - i; ++space) {
            cout << ' ';
        }

        for (int star = 1; star <= (2 * i - 1); ++star) {
            cout << '*';
        }

        cout << '\n';
    }

    return 0;
}
```

Listing 7: Centred star pyramid

A.4.5 Pattern formula

For row i in a pyramid of n rows:

$$\text{spaces} = n - i, \qquad \text{stars} = 2i - 1$$

Remember

Before coding a pattern, create a small row-wise table showing the row number, number of spaces and number of symbols. Derive the relationship first; then write the loops.

A.4.6 Practice extensions

1. Print an inverted triangle.
2. Print Floyd's triangle.
3. Print a hollow rectangle.
4. Print a full diamond using an upper and lower half.

A.5 Program 4: Menu-Driven Calculator Using Functions

A.5.1 Problem statement

Develop a menu-driven calculator that performs addition, subtraction, multiplication and division. Implement each operation in a separate function.

A.5.2 Concepts covered

- Function declaration, definition and call
- Parameters and return values
- `switch` statement
- Repetition using `do-while`
- Input validation and division-by-zero handling
- Modular procedural design

```cpp
#include <iostream>
using namespace std;

double add(double a, double b) {
    return a + b;
}

double subtract(double a, double b) {
    return a - b;
}

double multiply(double a, double b) {
    return a * b;
}

bool divide(double a, double b, double &result) {
    if (b == 0.0) {
        return false;
    }
    result = a / b;
    return true;
}

int main() {
    int choice;
```

```
26
27      do {
28          cout << "\n===== CALCULATOR =====\n";
29          cout << "1. Addition\n";
30          cout << "2. Subtraction\n";
31          cout << "3. Multiplication\n";
32          cout << "4. Division\n";
33          cout << "5. Exit\n";
34          cout << "Enter choice: ";
35          cin >> choice;
36
37          if (choice >= 1 && choice <= 4) {
38              double x, y;
39              cout << "Enter two numbers: ";
40              cin >> x >> y;
41
42              switch (choice) {
43                  case 1:
44                      cout << "Result = " << add(x, y) << '\n';
45                      break;
46                  case 2:
47                      cout << "Result = " << subtract(x, y) << '\n';
48                      break;
49                  case 3:
50                      cout << "Result = " << multiply(x, y) << '\n';
51                      break;
52                  case 4: {
53                      double result;
54                      if (divide(x, y, result)) {
55                          cout << "Result = " << result << '\n';
56                      } else {
57                          cout << "Error: division by zero.\n";
58                      }
59                      break;
60                  }
61              }
62          } else if (choice != 5) {
63              cout << "Invalid choice. Try again.\n";
64          }
65
66      } while (choice != 5);
67
68      cout << "Calculator closed.\n";
```

```
69      return 0;
70  }
```

Listing 8: Menu-driven calculator using functions

A.5.3 Why does the division function return `bool`?

The function must communicate two facts: whether division succeeded and, when it succeeded, the calculated quotient. It returns `true` or `false`, while the reference parameter `result` carries the quotient. Reference parameters are covered later in Unit 1, so this line can be treated as a preview.

Example

Calling `divide(10, 2, result)` returns `true` and stores `5` in `result`. Calling `divide(10, 0, result)` returns `false` and avoids an invalid operation.

A.5.4 Practice extensions

1. Add modulus for integer operands.
2. Add power and square-root operations using `<cmath>`.
3. Store the last result and allow it to be reused.
4. Rewrite the calculator using function overloading after that topic is taught.

A.6 Program 5: Procedural Student Record Management System

A.6.1 Problem statement

Create a menu-driven program that stores student records and supports the following operations:

- add a record;
- display all records;
- search by roll number;
- update marks;
- delete a record.

A.6.2 Why this program matters

This program combines structures, arrays, functions, loops and menu-driven logic. More importantly, it exposes the weaknesses of a procedural design and creates the motivation for classes, objects and data hiding.

```
#include <iomanip>
#include <iostream>
#include <string>
using namespace std;

const int MAX_STUDENTS = 100;

struct Student {
    int rollNumber;
    string name;
    double marks;
};

int findStudent(const Student students[], int count, int rollNumber)
   {
    for (int i = 0; i < count; ++i) {
        if (students[i].rollNumber == rollNumber) {
            return i;
        }
    }
    return -1;
}

void addStudent(Student students[], int &count) {
```

```
24      if (count == MAX_STUDENTS) {
25          cout << "Storage is full.\n";
26          return;
27      }
28
29      Student newStudent;
30      cout << "Enter roll number: ";
31      cin >> newStudent.rollNumber;
32
33      if (findStudent(students, count, newStudent.rollNumber) != -1) {
34          cout << "A student with this roll number already exists.\n";
35          return;
36      }
37
38      cin.ignore();
39      cout << "Enter name: ";
40      getline(cin, newStudent.name);
41      cout << "Enter marks: ";
42      cin >> newStudent.marks;
43
44      students[count] = newStudent;
45      ++count;
46      cout << "Record added successfully.\n";
47  }
48
49  void displayStudents(const Student students[], int count) {
50      if (count == 0) {
51          cout << "No records available.\n";
52          return;
53      }
54
55      cout << left << setw(12) << "Roll No."
56           << setw(25) << "Name"
57           << setw(10) << "Marks" << '\n';
58      cout << string(47, '-') << '\n';
59
60      for (int i = 0; i < count; ++i) {
61          cout << left << setw(12) << students[i].rollNumber
62               << setw(25) << students[i].name
63               << setw(10) << students[i].marks << '\n';
64      }
65  }
66
```

```
67 void searchStudent(const Student students[], int count) {
68     int rollNumber;
69     cout << "Enter roll number to search: ";
70     cin >> rollNumber;
71 
72     int index = findStudent(students, count, rollNumber);
73     if (index == -1) {
74         cout << "Record not found.\n";
75         return;
76     }
77 
78     cout << "Name  : " << students[index].name << '\n';
79     cout << "Marks : " << students[index].marks << '\n';
80 }
81 
82 void updateStudent(Student students[], int count) {
83     int rollNumber;
84     cout << "Enter roll number to update: ";
85     cin >> rollNumber;
86 
87     int index = findStudent(students, count, rollNumber);
88     if (index == -1) {
89         cout << "Record not found.\n";
90         return;
91     }
92 
93     cout << "Enter new marks: ";
94     cin >> students[index].marks;
95     cout << "Record updated successfully.\n";
96 }
97 
98 void deleteStudent(Student students[], int &count) {
99     int rollNumber;
100    cout << "Enter roll number to delete: ";
101    cin >> rollNumber;
102 
103    int index = findStudent(students, count, rollNumber);
104    if (index == -1) {
105        cout << "Record not found.\n";
106        return;
107    }
108 
109    for (int i = index; i < count - 1; ++i) {
```

```
110         students[i] = students[i + 1];
111     }
112
113     --count;
114     cout << "Record deleted successfully.\n";
115 }
116
117 int main() {
118     Student students[MAX_STUDENTS];
119     int count = 0;
120     int choice;
121
122     do {
123         cout << "\n===== STUDENT RECORD SYSTEM =====\n";
124         cout << "1. Add record\n";
125         cout << "2. Display all records\n";
126         cout << "3. Search record\n";
127         cout << "4. Update marks\n";
128         cout << "5. Delete record\n";
129         cout << "6. Exit\n";
130         cout << "Enter choice: ";
131         cin >> choice;
132
133         switch (choice) {
134             case 1:
135                 addStudent(students, count);
136                 break;
137             case 2:
138                 displayStudents(students, count);
139                 break;
140             case 3:
141                 searchStudent(students, count);
142                 break;
143             case 4:
144                 updateStudent(students, count);
145                 break;
146             case 5:
147                 deleteStudent(students, count);
148                 break;
149             case 6:
150                 cout << "Program terminated.\n";
151                 break;
152             default:
```

```
153                 cout << "Invalid choice. Try again.\n";
154             }
155         } while (choice != 6);
156
157         return 0;
158 }
```

Listing 9: Procedural student record management system

A.6.3 Design limitations observed

1. The array and record count are managed separately and must always remain consistent.
2. Any function receiving a non-constant array can directly change record data.
3. Validation rules are scattered across functions.
4. Data and operations are related conceptually but remain physically separated.
5. Adding attendance, address, semester or multiple subjects makes the program increasingly difficult to maintain.

A.6.4 OOP transition preview

The procedural structure

```
Student data + separate functions
```

will later become

```
class Student {
    private data
    public operations
};
```

Remember

A `struct` is not automatically procedural and a `class` is not automatically good OOP. The quality of a design depends on responsibility assignment, controlled access and meaningful abstraction.

A.7 Combined Practice Sheet

A.7.1 Short-answer questions

1. Why is `double` preferable to `int` for percentage and bill calculations?
2. What is the purpose of `return 0;` in `main()`?
3. Differentiate between a syntax error, runtime error and logical error.
4. Why are nested loops required for two-dimensional patterns?
5. What is the benefit of dividing a calculator into separate functions?
6. Why must division by zero be checked explicitly?
7. What value does `findStudent()` return when no record is found?
8. Why are the array parameters marked `const` in read-only functions?

A.7.2 Debugging exercises

Find and correct the error in each statement.

```
cout >> "Enter marks: ";
cin << marks;
if (percentage = 90)
for (int i = 1; i < = rows; i++)
if (choice == 4 && divisor = 0)
```

A.7.3 Programming assignments

1. Modify Program 1 to calculate the highest and lowest marks.
2. Modify Program 2 to support a commercial tariff with different slabs.
3. Print Pascal's triangle up to n rows.
4. Extend Program 4 with power, factorial and percentage operations.
5. Add sorting by roll number to Program 5.
6. Convert Program 5 into an object-oriented design after classes and objects are taught.

Takeaway

The purpose of these programs is not to memorise code. The purpose is to develop the ability to convert a problem statement into input, processing, decisions, repetition, functions and organised data. That foundation is essential before students can use object-oriented features correctly.